Y8-ProtoMav CPU Manual

By: Orest Shkaba

I pledge my honor that I have abided by the Stevens Honor System.

How To Use:

Step 1: Create an assembly file. The first line of this must begin with DATA followed by 1 to 16 values. These values can be written in decimal between 0-255, or as hexadecimals from 0x0-0xFF. These values will be stored in Data RAM addresses 0-15 in the order written.

Step 2: Below the DATA line, write exactly one instruction per line using the formats ADD Rd Rn Rt, MULT Rd Rn Rt, LD Rt Memory, or STR Rt Memory. The CPU has 4 2-bit registers represented as 0-3 decimal values, and all addresses must be integers between 0-15. Comments can be used with # where everything after is neglected in runtime. (More Information about instructions can be found on page 2).

Step 3: Save the assembly file in the same directory as assembler.py and run in the terminal “python3 assembler.py <file_name>” to generate the files instr.o and data.o. (Any file name extension suffices)

Step 4: Open the CPU in Logisim and use “Load image…” to load instr.o into the Instruction Memory RAM and data.o into the Data RAM.

Step 5: Reset the simulation to make Program Counter 0 and press the clock button to execute the program. Each tick fetches the next instruction from the instruction RAM, executes it, and updates the registers or Data RAM if required. The Program Counter indicates the address of the instruction being executed.

Architecture Description of CPU:

The CPU has 4 general purpose registers (R0-R3) represented as 0-3 in decimal where each register can store a 2 digit hexadecimal number, or 8 bits from 0-255 in decimal.

CPU Functions:

1. ADD - instruction adds the values of two registers and stores the sum to a register.
2. MULT - instruction multiplies the values of two registers and stores the product to a register.
3. LD - instruction gets a value from a memory address in RAM and stores it to a register.
4. STR - instruction gets a value from a register and stores it in a memory address.

Instructions Available:

Instruction	General Format	Opcode (Bits 7-6)	Register Field (Bits 5-4)	Remaining Field (Bits 3-0)
ADD	ADD Rd Rn Rt	00	Rd (2 bits)	Rn and Rt (2 bits each)
MULT	MULT Rd Rn Rt	01	Rd (2 bits)	Rn and Rt (2 bits each)
LD	LD Rt Memory	10	Rt (2 bits)	M[3:0] (4-bit data address)
STR	STR Rt Memory	11	Rt (2 bits)	M[3:0] (4-bit data address)

The opcode field uses 2 bits because the CPU has 4 instructions and $2^2 = 4$ so 4 distinct opcodes are needed. Furthermore, each register field uses 2 bits because there are 4 general purpose registers and $2^2 = 4$ patterns are needed to select them. Lastly, the memory address field uses 4 bits because data memory has 16 locations of addresses 0-15 and $2^4 = 16$ patterns are needed to represent all possible addresses.